The Problem of Maximum Flow in Networks

Przemysław Gordinowicz

Institute of Mathematics,
Łódź University of Technology

Graphs & Social Networks
Lecture 9

Outline

Outline

Linear Programming (LP)

Kantorowicz [1940], Dantzig [1947]

Given $c \in \mathbb{R}^n$, $b \in \mathbb{R}^m$, $A \in \mathcal{M}_{m,n}$, $m \leq n$ find $x \in \mathbb{R}^n$ such that

$$\begin{aligned} \text{maximize} \quad & c^T x \\ \text{satisfying} \quad & Ax \leq b, \\ & x \geq 0. \end{aligned}$$

The found x is called *optimal solution*, any x satisfying both conditions — *feasible solution*.

Other forms (minimization, $Ax = b$, $Ax \geq b$, abandoning $x \geq 0$) are easy to obtain by simple (clever) transformations.

Linear Programming (LP)

Kantorowicz [1940], Dantzig [1947]

Given $c \in \mathbb{R}^n, b \in \mathbb{R}^m, A \in \mathcal{M}_{m,n}$, $m \leq n$ find $x \in \mathbb{R}^n$ such that

$$\begin{aligned} \text{maximize} \quad & c^T x \\ \text{satisfying} \quad & Ax \leq b, \\ & x \geq 0. \end{aligned}$$

The found x is called *optimal solution*, any x satisfying both conditions — *feasible solution*.
Other forms (minimization, $Ax = b$, $Ax \geq b$, abandoning $x \geq 0$) are easy to obtain by simple (clever) transformations.

Linear Programming (LP)

Kantorowicz [1940], Dantzig [1947]

Given $c \in \mathbb{R}^n, b \in \mathbb{R}^m, A \in \mathcal{M}_{m,n}$, $m \leq n$ find $x \in \mathbb{R}^n$ such that

$$\begin{aligned} \text{maximize} \quad & c^T x \\ \text{satisfying} \quad & Ax \leq b, \\ & x \geq 0. \end{aligned}$$

The found x is called *optimal solution*, any x satisfying both conditions — *feasible solution*.
There are efficient algorithms solving LPs, starting from the **simplex algorithm** (thanks to Dantzig [1947], improved later, although of exponential complexity, in theory). Huge progress recently (fast polynomial algorithms).

Duality (von Neumann, ca. 1950)

$$\begin{aligned} \text{maximize} \quad & c^T x \\ \text{satisfying} \quad & Ax \leq b, x \geq 0. \end{aligned}$$

$$\begin{aligned} \text{minimize} \quad & \lambda^T b \\ \text{satisfying} \quad & \lambda^T A \geq c^T, \lambda \geq 0. \end{aligned}$$

Theorem (Weak duality theorem)

If both problems have feasible solutions x and λ, then $c^T x \leq \lambda^T b$.

Therefore, if one of them is unbounded, then the other has no feasible solution and $\max c^T x \leq \min \lambda^T b$.

Duality (von Neumann, ca. 1950)

$$\begin{aligned} \text{maximize} \quad & c^T x \\ \text{satisfying} \quad & Ax \leq b, x \geq 0. \end{aligned}$$

$$\begin{aligned} \text{minimize} \quad & \lambda^T b \\ \text{satisfying} \quad & \lambda^T A \geq c^T, \lambda \geq 0. \end{aligned}$$

Theorem (Weak duality theorem)

If both problems have feasible solutions x and λ, then $c^T x \leq \lambda^T b$.

Therefore, if one of them is unbounded, then the other has no feasible solution and $\max c^T x \leq \min \lambda^T b$.

Duality (von Neumann, ca. 1950)

$$\begin{aligned} \text{maximize} \quad & c^T x \\ \text{satisfying} \quad & Ax \leq b, x \geq 0. \end{aligned}$$

$$\begin{aligned} \text{minimize} \quad & \lambda^T b \\ \text{satisfying} \quad & \lambda^T A \geq c^T, \lambda \geq 0. \end{aligned}$$

Theorem (Strong duality theorem)

If one of the problems has a finite optimal solution, then the other also has one and $\max c^T x = \min \lambda^T b$.

Integer linear programming (ILP)

Given $c \in \mathbb{R}^n, b \in \mathbb{R}^m, A \in \mathcal{M}_{m,n}$, $m \leq n$ find $x \in \mathbb{Z}^n$ such that

$$\begin{aligned} \text{maximize} \quad & c^T x \\ \text{satisfying} \quad & Ax \leq b, \\ & x \geq 0. \end{aligned}$$

Simple example: withdrawing a given amount from an ATM with the smallest number of banknotes.

Integer linear programming (ILP)

Given $c \in \mathbb{R}^n, b \in \mathbb{R}^m, A \in \mathcal{M}_{m,n}$, $m \leq n$ find $x \in \mathbb{Z}^n$ such that

$$\begin{aligned} \text{maximize} \quad & c^T x \\ \text{satisfying} \quad & Ax \leq b, \\ & x \geq 0. \end{aligned}$$

Simple example: withdrawing a given amount from an ATM with the smallest number of banknotes.

Integer linear programming (ILP)

Given $c \in \mathbb{R}^n, b \in \mathbb{R}^m, A \in \mathcal{M}_{m,n}$, $m \leq n$ find $x \in \mathbb{Z}^n$ such that

$$\begin{aligned} \text{maximize} \quad & c^T x \\ \text{satisfying} \quad & Ax \leq b, \\ & x \geq 0. \end{aligned}$$

Question: compared to a regular LP, is this problem easier, more difficult, or the same?

Integer linear programming (ILP)

Given $c \in \mathbb{R}^n, b \in \mathbb{R}^m, A \in \mathcal{M}_{m,n}$, $m \leq n$ find $x \in \mathbb{Z}^n$ such that

$$\begin{aligned} \text{maximize} \quad & c^T x \\ \text{satisfying} \quad & Ax \leq b, \\ & x \geq 0. \end{aligned}$$

Question: compared to a regular LP, is this problem easier, more difficult, or the same?

In fact this problem is *NP*-hard (TSP problem may be presented as an instance of ILP).

Illustration — Let's Pack a Knapsack!

Given $c \in \mathbb{R}^n$, $w \in \mathbb{R}^n$, $q \in \mathbb{R}^n$ and $B \in \mathbb{R}$

$$\begin{aligned} \text{maximize} \quad & c^T x = \sum_{i=1}^{n} c_i x_i \\ \text{satisfying} \quad & w^T x = \sum_{i=1}^{n} w_i x_i \leq B, \ \text{(fit in the knapsack)} \\ & x \leq q, \ \text{(availability)} \\ & x \geq 0. \end{aligned}$$

When $x \in \mathbb{R}^n$ (we pack divisible things) this is a *continuous knapsack problem* — uninteresting.
When $x \in \mathbb{Z}^n$ (we pack indivisible things) this is a (usual) **knapsack problem** (although sometimes it is restricted to $x \in \{0,1\}^n$).

Optimal Assignment

For a given matrix $\mathcal{W} \in M_{n,n}$ find the **transversal** with the largest value.

$$\mathcal{A} = \begin{bmatrix} 4 & 1 & 6 & 2 & 3 \\ 5 & 0 & 3 & 7 & 6 \\ 2 & 3 & 4 & 5 & 8 \\ 3 & 4 & 6 & 3 & 4 \\ 4 & 6 & 5 & 8 & 6 \end{bmatrix}$$

Optimal Assignment

For a given matrix $\mathcal{W} \in M_{n,n}$ find the **transversal** with the largest value.

$$\mathcal{A} = \begin{bmatrix} 4 & 1 & 6 & 2 & 3 \\ 5 & 0 & 3 & 7 & 6 \\ 2 & 3 & 4 & 5 & 8 \\ 3 & 4 & 6 & 3 & 4 \\ 4 & 6 & 5 & 8 & 6 \end{bmatrix}$$

Optimal Assignment

For a given matrix $\mathcal{W} \in M_{n,n}$ find the **transversal** with the largest value.

$$\mathcal{A} = \begin{bmatrix} 4 & 1 & 6 & 2 & 3 \\ 5 & 0 & 3 & 7 & 6 \\ 2 & 3 & 4 & 5 & 8 \\ 3 & 4 & 6 & 3 & 4 \\ 4 & 6 & 5 & 8 & 6 \end{bmatrix}$$

Discussed in the last lecture (and labs).
An example of an integer (binary) program that has an elegant, efficient, combinatorial algorithm for solving it.

Outline

Motivation

Ford and Fulkerson (who [1956] formulated the **maximum flow** problem and provided the first methods for its solution) concluded (book, [1968]) that over 90% of linear programming problems solved for industrial needs (including logistics) are the maximum flow problem and related problems.

Thanks to such a combinatorial representation of this problem, it can be (it could have been) solved more efficiently.

Motivation

Ford and Fulkerson (who [1956] formulated the **maximum flow** problem and provided the first methods for its solution) concluded (book, [1968]) that over 90% of linear programming problems solved for industrial needs (including logistics) are the maximum flow problem and related problems.

Thanks to such a combinatorial representation of this problem, it can be (it could have been) solved more efficiently.

Flow networks

Considering functions of the form $f\colon V \times V \to \mathbb{R}$, with vertices as arguments, their value will be written as $f_{uv} = f(u, v)$.
We will also use the **implicit summation notation**, where the arguments are sets of vertices. For example, for $X, Y \subseteq V$

$$f(X, Y) = \sum_{x \in X} \sum_{y \in Y} f_{xy}$$

Definition

A flow network is an ordered quadruple $N = (G, s, t, c)$, where $G = (V, A)$ is a simple digraph, **source** $s \in V$ and **sink** $t \in V$ are two distinct vertices, and c is a **capacity** function $c\colon V \times V \to \mathbb{R}_+ \cup \{0\}$ satisfying the condition $c_{uv} > 0 \Rightarrow (u, v) \in A$.

In fact this is a weighted digraph with two distinguished vertices.

Flow networks

Considering functions of the form $f\colon V \times V \to \mathbb{R}$, with vertices as arguments, their value will be written as $f_{uv} = f(u, v)$.
We will also use the **implicit summation notation**, where the arguments are sets of vertices. For example, for $X, Y \subseteq V$

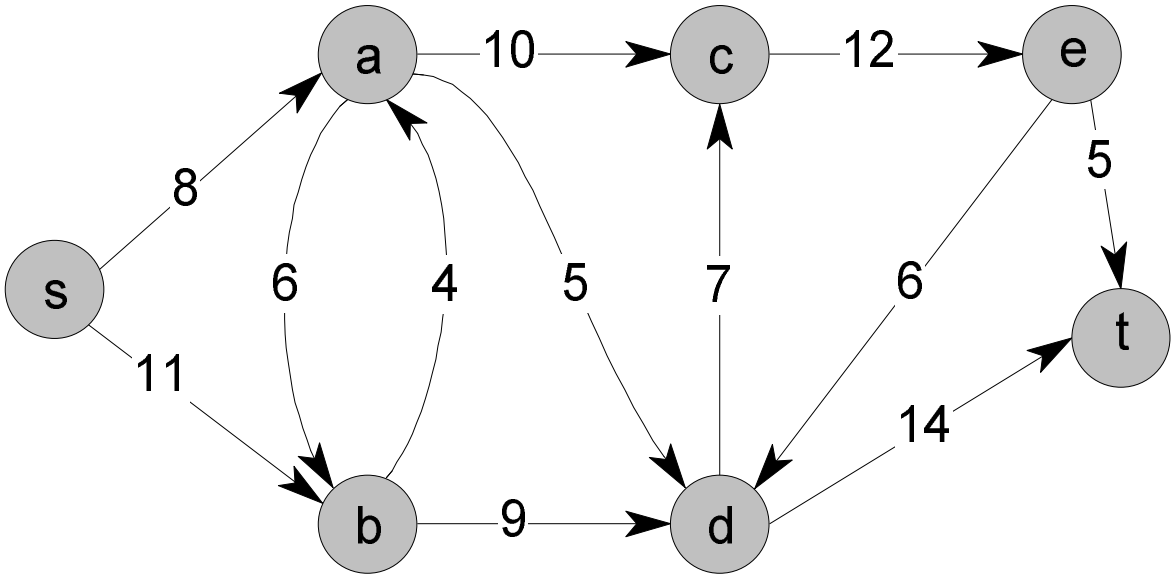
$$f(X, Y) = \sum_{x \in X} \sum_{y \in Y} f_{xy}$$

Definition

A flow network is an ordered quadruple $N = (G, s, t, c)$, where $G = (V, A)$ is a simple digraph, **source** $s \in V$ and **sink** $t \in V$ are two distinct vertices, and c is a **capacity** function $c\colon V \times V \to \mathbb{R}_+ \cup \{0\}$ satisfying the condition $c_{uv} > 0 \Rightarrow (u, v) \in A$.

In fact this is a weighted digraph with two distinguished vertices.

Flow networks



Networks Flows. Maximum Flow

Definition

A flow in the network N is any function $f\colon V \times V \to \mathbb{R}$ that satisfies the following conditions:

- **capacity:** $\forall u, v \in V : \quad f_{uv} \leq c_{uv}$
- **skew symmetry:** $\forall u, v \in V : \quad f_{uv} = -f_{vu}$
- **flow conservation:**

$$\forall u \in V \setminus \{s, t\} : \quad \sum_{v \in V} f_{uv} = 0$$

$$\text{moreover} \quad \sum_{v \in V} f_{sv} \geq 0 \quad \text{and} \quad \sum_{v \in V} f_{vt} \geq 0$$

The value of the flow f is: $\mathcal{F} = \sum_{v \in V} f_{vt}$.
The flow with maximum value is called **maximum flow.**

Network Flow — simple properties

Lemma

Let N be a flow network with source s and sink t. Let f be a flow in this network of value $\mathcal{F}$. Then, for $X, Y, Z \subset V$, such that $X \cap Y = \emptyset$ the following equalities holds:

a) $f(X, Z) = -f(Z, X)$

b) $f(X, X) = 0$

c) $f(X \cup Y, Z) = f(X, Z) + f(Y, Z)$

d) $f(\{s\}, V) = f(V, \{t\}) = \mathcal{F}$

Definition

If f is a flow in some network N, then a **positive flow** is the function $f^+ : V \times V \to \mathbb{R}$ such that $f^+_{uv} = \max\{f_{uv}, 0\}$.

From the condition of skew symmetry it follows that a positive flow uniquely determines a flow in the network.

Network Flow — simple properties

Lemma

Let N be a flow network with source s and sink t. Let f be a flow in this network of value $\mathcal{F}$. Then, for $X, Y, Z \subset V$, such that $X \cap Y = \emptyset$ the following equalities holds:

a) $f(X, Z) = -f(Z, X)$

b) $f(X, X) = 0$

c) $f(X \cup Y, Z) = f(X, Z) + f(Y, Z)$

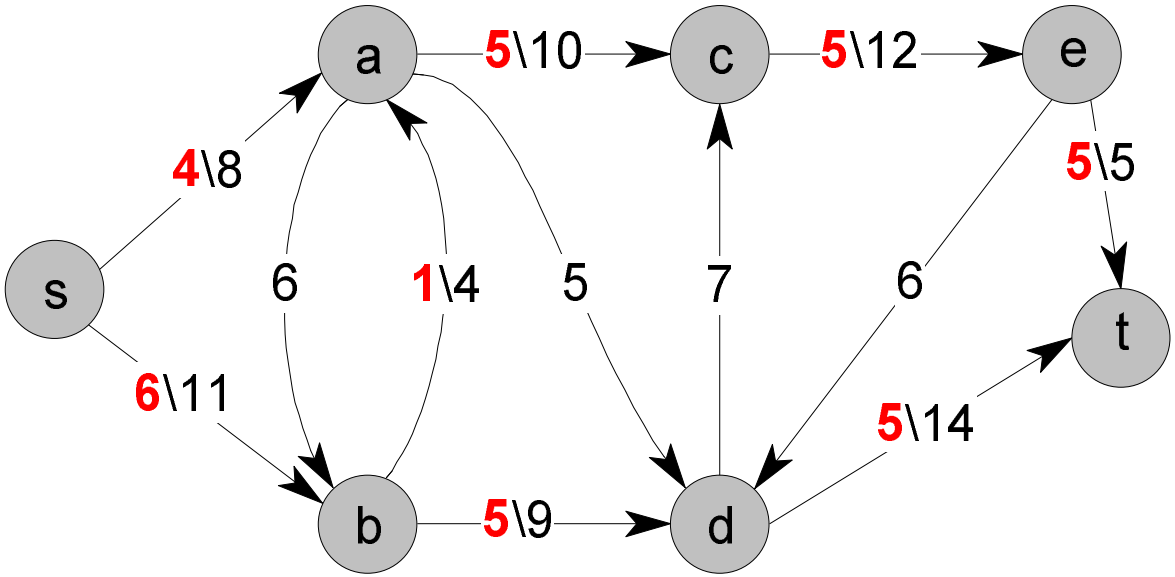
d) $f(\{s\}, V) = f(V, \{t\}) = \mathcal{F}$

Definition

If f is a flow in some network N, then a **positive flow** is the function $f^+ : V \times V \to \mathbb{R}$ such that $f^+_{uv} = \max\{f_{uv}, 0\}$.

From the condition of skew symmetry it follows that a positive flow uniquely determines a flow in the network.

Network Flow — simple properties



Digression — another type of network flow

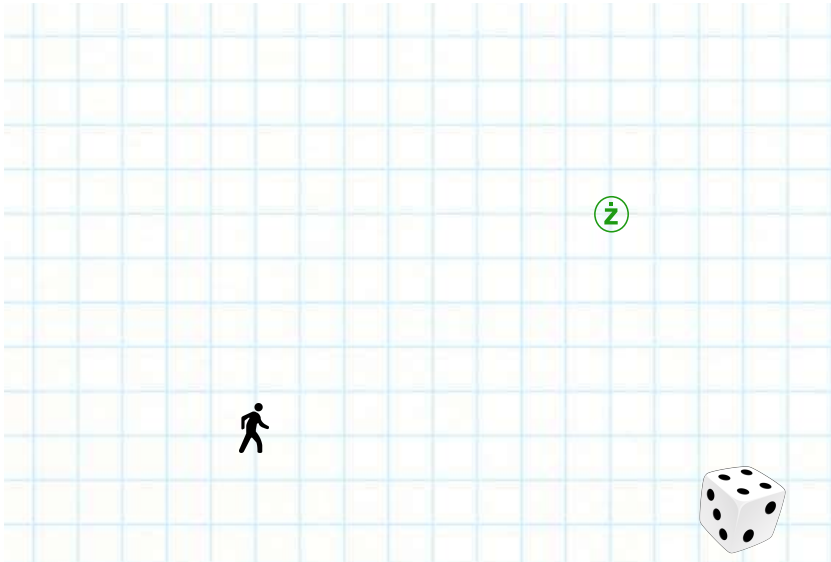
This approach (with capacity conditions) is mostly inspired by the logistic applications.

However, there are analysed as well, flows in electrical networks, with a resistance conditions instead. They have surprising applications in graph random walks analysis.

Digression — another type of network flow

This approach (with capacity conditions) is mostly inspired by the logistic applications.

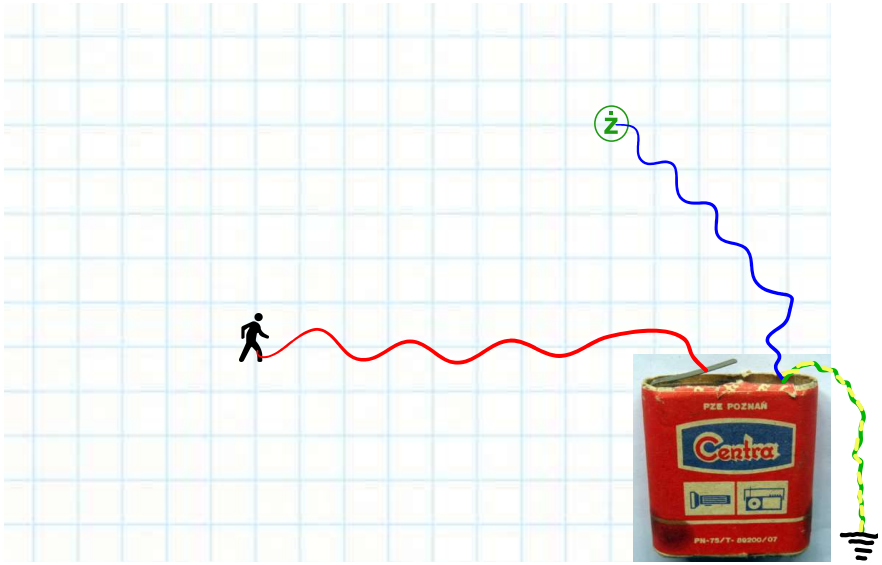
However, there are analysed as well, flows in electrical networks, with a resistance conditions instead. They have surprising applications in graph random walks analysis.



Digression — another type of network flow

This approach (with capacity conditions) is mostly inspired by the logistic applications.

However, there are analysed as well, flows in electrical networks, with a resistance conditions instead. They have surprising applications in graph random walks analysis.



Residual network

Definition

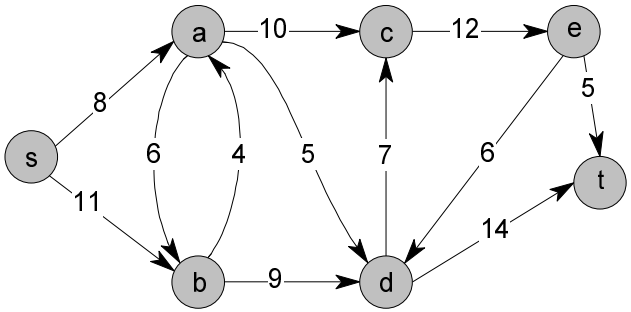
Let $G = (V, A)$ form a flow network, with capacity function c, source s and sink t, let f be some flow in this network.
Residual capacity is the function $c^f : V \times V \to \mathbb{R}$, such that:

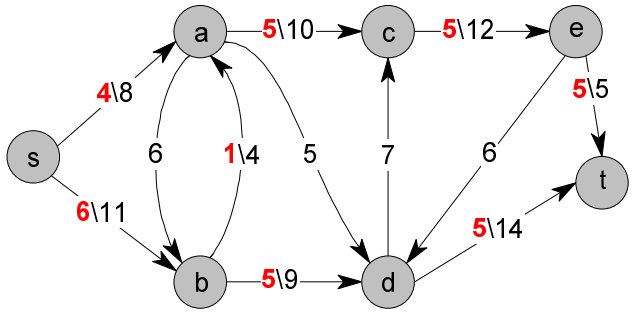
$$c^f_{uv} = c_{uv} - f_{uv}.$$

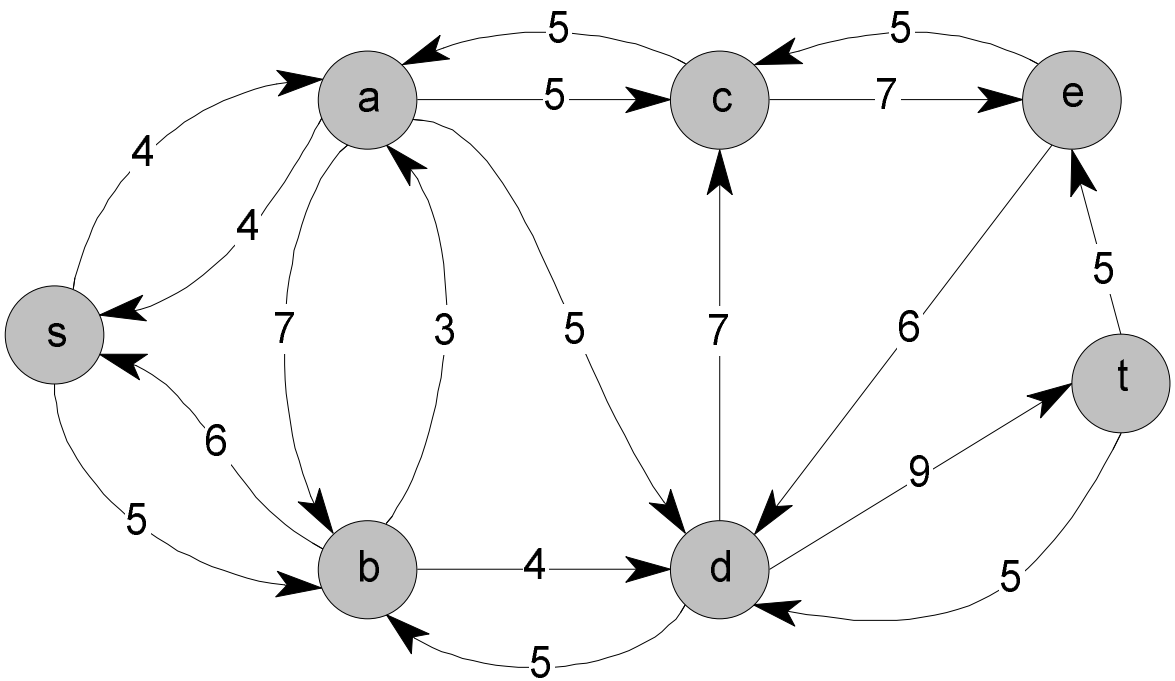
A residual network induced by a flow f is a flow network $N^f = (G^f, s, t, c^f)$, $G^f = (V, A^f)$, where $A^f = \{(u, v) \in V \times V : c^f_{uv} > 0\}$.

The arc (u, v) can appear in the residual network only when $(u, v) \in A$ or $(v, u) \in A$.

Residual network







Augmenting paths

Lemma

Let N be a flow network, f — some flow in this network, N^f — a residual network of N induced by f, and f' — a flow in N^f. Then the sum of flows $f + f'$ (in the sense of a sum of functions) is a flow in N with value $|f + f'| = \mathcal{F} + \mathcal{F}'$.

Definition

Let N be a flow network, and f some flow in this network. An **augmenting path** p is any path from s to t in N^f. The largest possible flow along the edges of this path is called its **residual capacity** and is defined as $c^f(p) = \min\{c^f_{uv} : (u, v) \in p\}$.

The residual capacity of any augmenting path is positive because all arcs in the residual network have positive capacities.

Augmenting paths

Lemma

Let N be a flow network, f — some flow in this network, N^f — a residual network of N induced by f, and f' — a flow in N^f. Then the sum of flows $f + f'$ (in the sense of a sum of functions) is a flow in N with value $|f + f'| = \mathcal{F} + \mathcal{F}'$.

Definition

Let N be a flow network, and f some flow in this network. An **augmenting path** p is any path from s to t in N^f. The largest possible flow along the edges of this path is called its **residual capacity** and is defined as $c^f(p) = \min\{c^f_{uv} : (u, v) \in p\}$.

The residual capacity of any augmenting path is positive because all arcs in the residual network have positive capacities.

Duality or the minimum cut

Definition

Let $G = (V, A)$ form a flow network, with capacity function c, source s and sink t. Each partition of the set V into sets S and $T = V \setminus S$ such that $s \in S$ and $t \in T$, is called a **cut** (S, T) in the network.
The **capacity** of a cut S, T is the sum $c(S, T)$. If f is a flow, then the **flow through the cut** (S, T) is defined as $f(S, T)$.

Lemma

Let f be a flow of value $\mathcal{F}$ in network N and let (S, T) be an arbitrary cut in this network. The flow through (S, T) is then $f(S, T) = \mathcal{F}$.

Max-flow Min-cut Theorem

Theorem

If f is a flow in network N, then the following conditions are equivalent:

1. *Flow f is a maximum flow.*
2. *The residual network N^f contains no augmenting paths.*
3. *For some cut (S, T), there is $\mathcal{F} = c(S, T)$.*

Algorithms

Most algorithms for determining maximum flow are based on one of the three methods:

- Linear programming
- Ford-Fulkerson method — augmenting flow along a single path
- Dinitz's method — processing a "bunch" of all shortest augmenting paths at once
- Karzanov's method — pushing the preflow with different control methods

Algorithms

Most algorithms for determining maximum flow are based on one of the four methods:

- Linear programming
- Ford-Fulkerson method — augmenting flow along a single path
- Dinitz's method — processing a "bunch" of all shortest augmenting paths at once
- Karzanov's method — pushing the preflow with different control methods

Ford-Fulkerson method

Function Flow_FF(G, s, t, c);
    $f := 0$;
    while there is a path $s \longrightarrow t$ in residual network **do**
        $p :=$ augmenting path;
        $c :=$ capacity of path p;
        augment f along path p (o c);
    Flow_FF $:= f$;

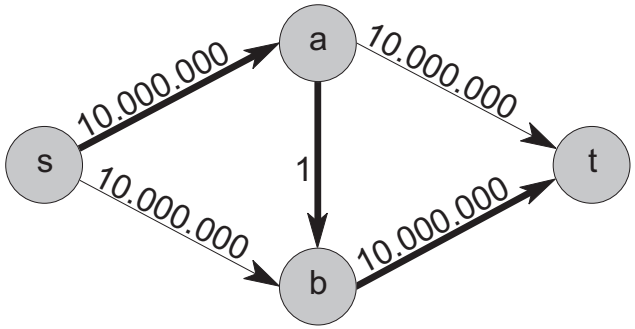
It is a good idea to choose the shortest path $s \longrightarrow t$ as the augmenting path (i.e. a path given by breadth-first search) — this is what the Edmonds-Karp algorithm [1972] does.

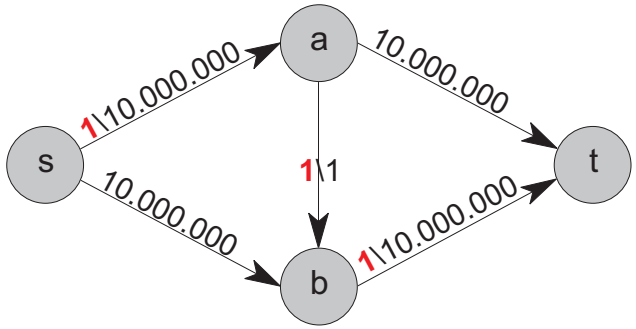
Ford-Fulkerson method

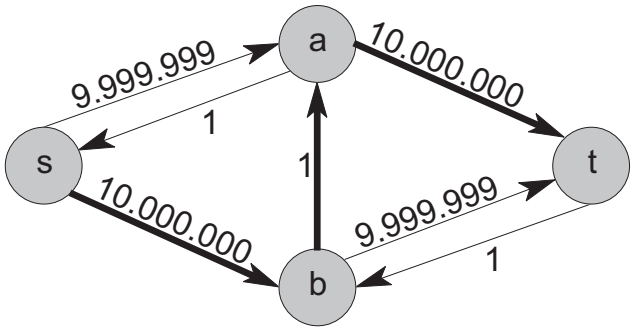
Function Flow_FF(G, s, t, c);
  $f := 0$;
  while there is a path $s \longrightarrow t$ in residual network **do**
    $p :=$ augmenting path;
    $c :=$ capacity of path p;
    augment f along path p (o c);
  Flow_FF $:= f$;

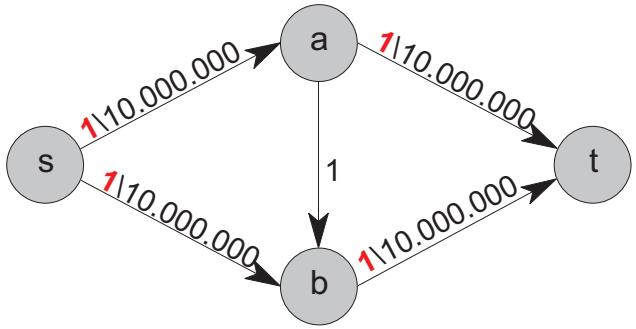
It is a good idea to choose the shortest path $s \longrightarrow t$ as the augmenting path (i.e. a path given by breadth-first search) — this is what the Edmonds-Karp algorithm [1972] does.

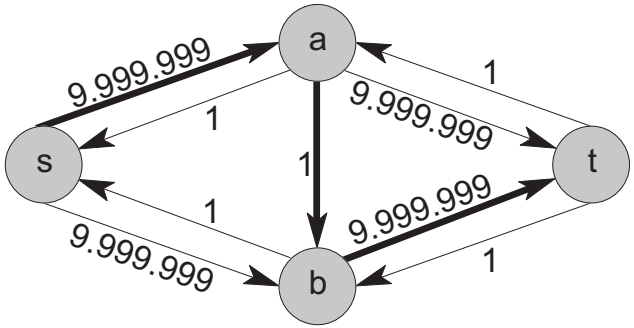
Ford-Fulkerson method — “bad example”

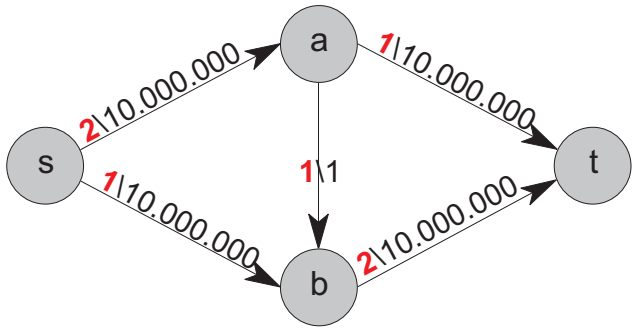




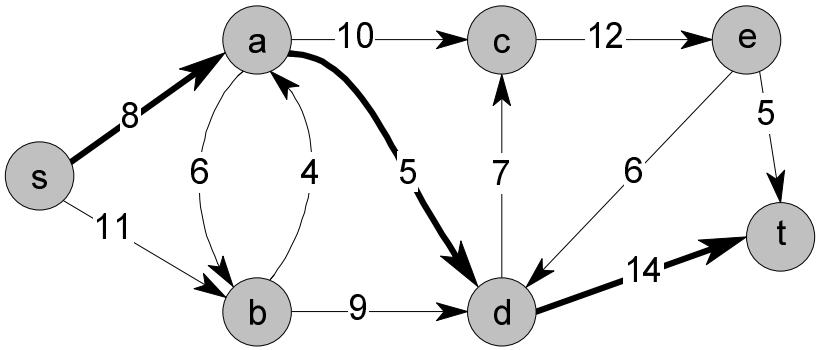


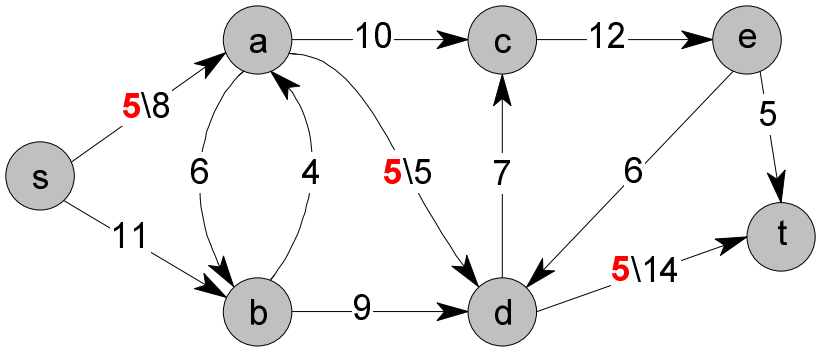


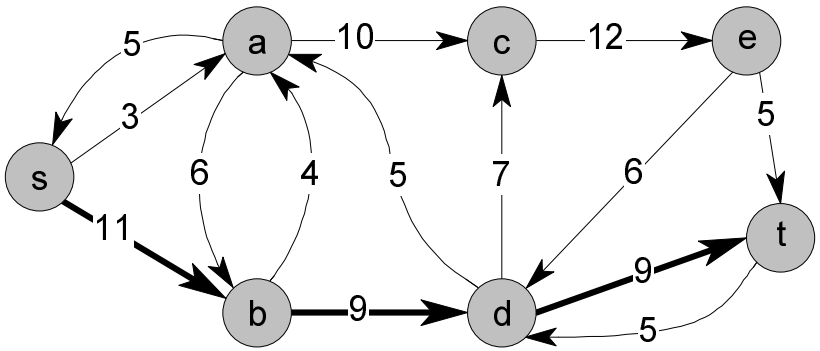


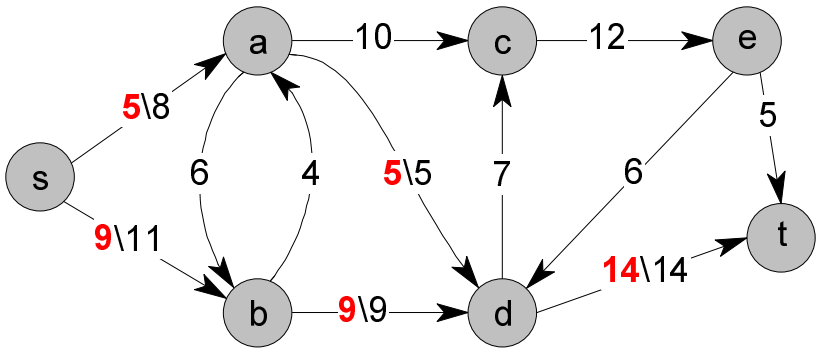


Edmonds-Karp algorithm — an example

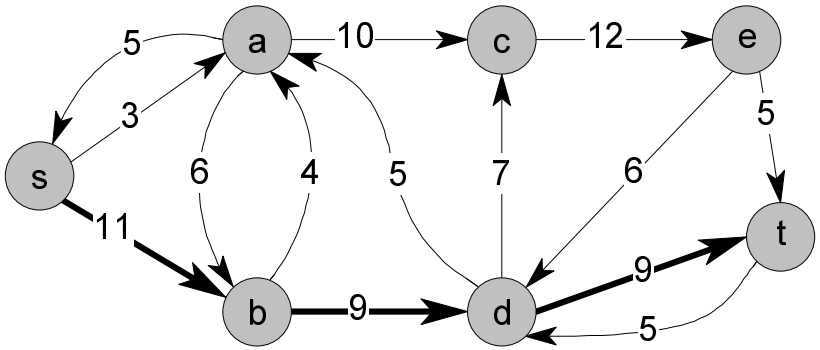


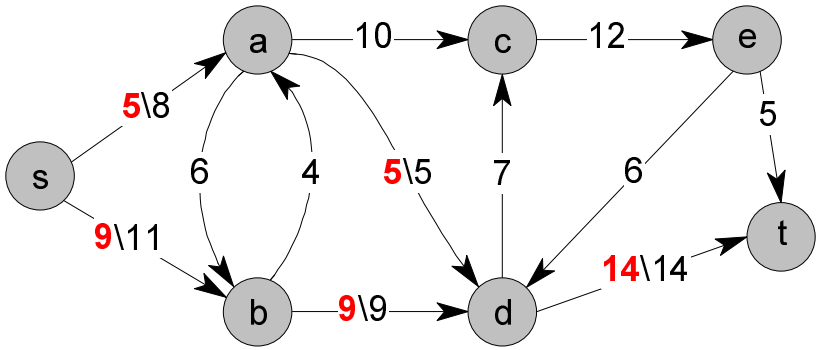


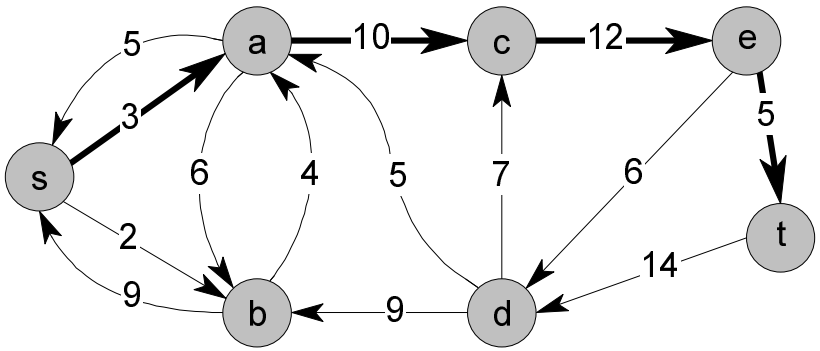


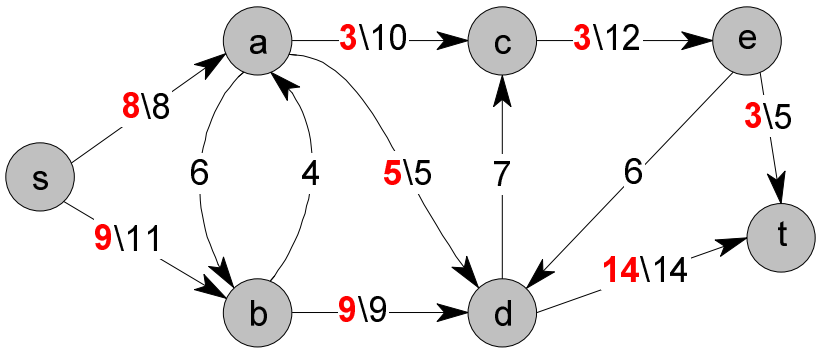


Edmonds-Karp algorithm — an example

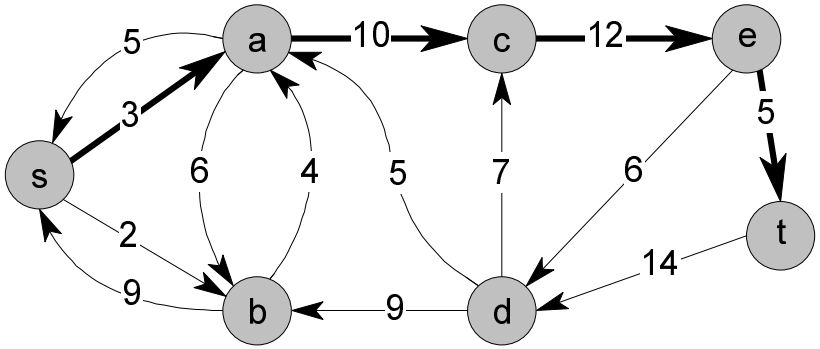


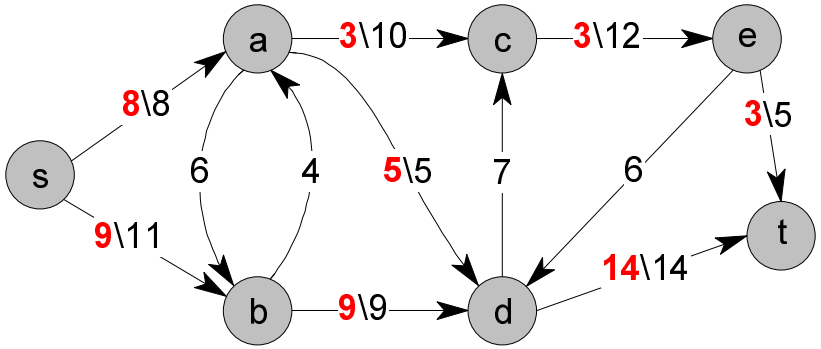


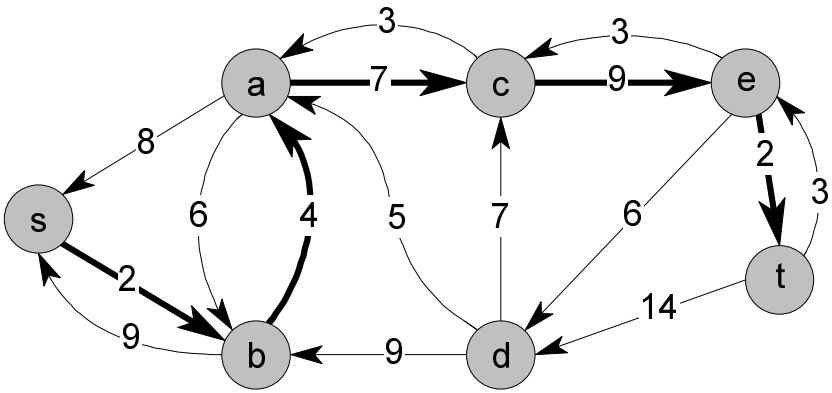


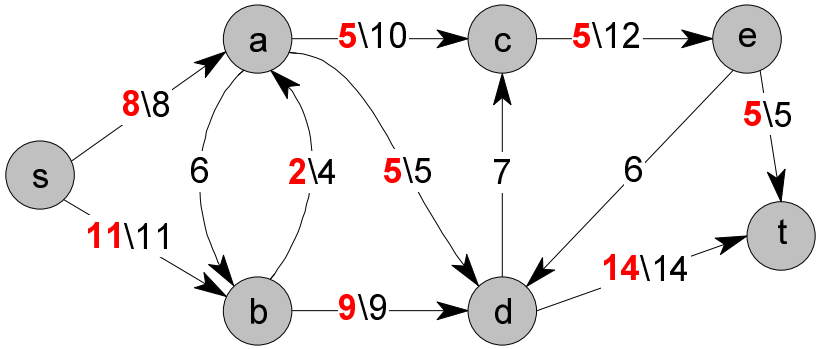


Edmonds-Karp algorithm — an example

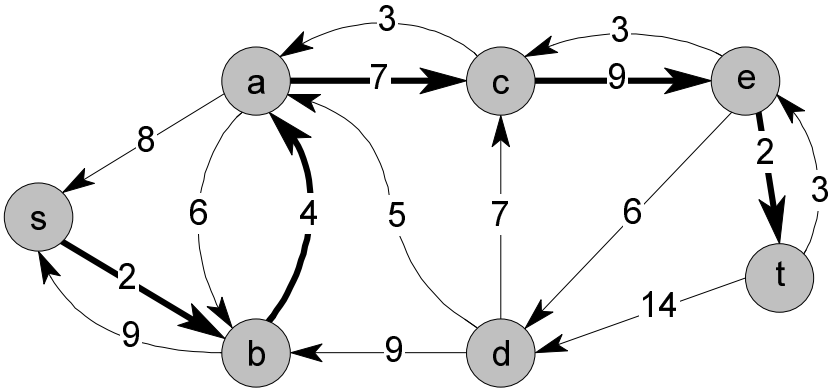


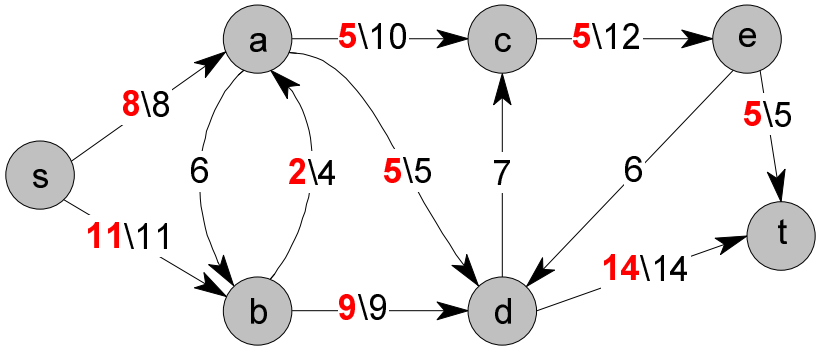


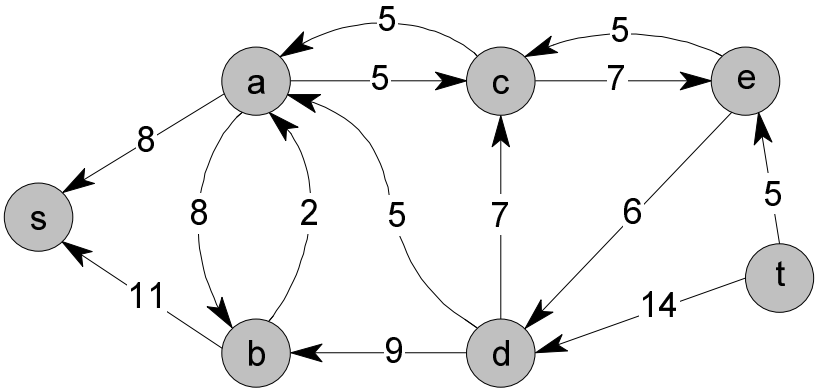




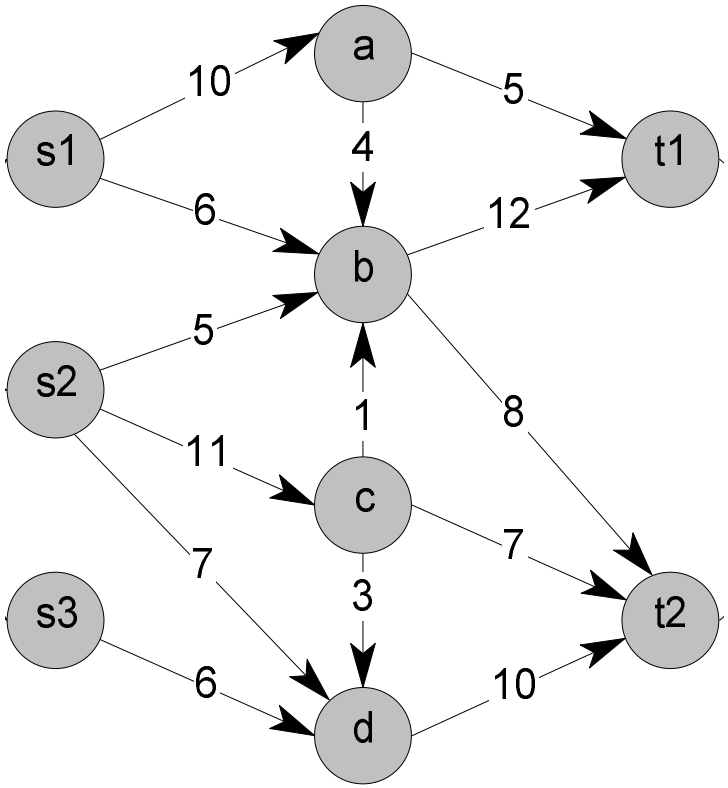
Edmonds-Karp algorithm — an example



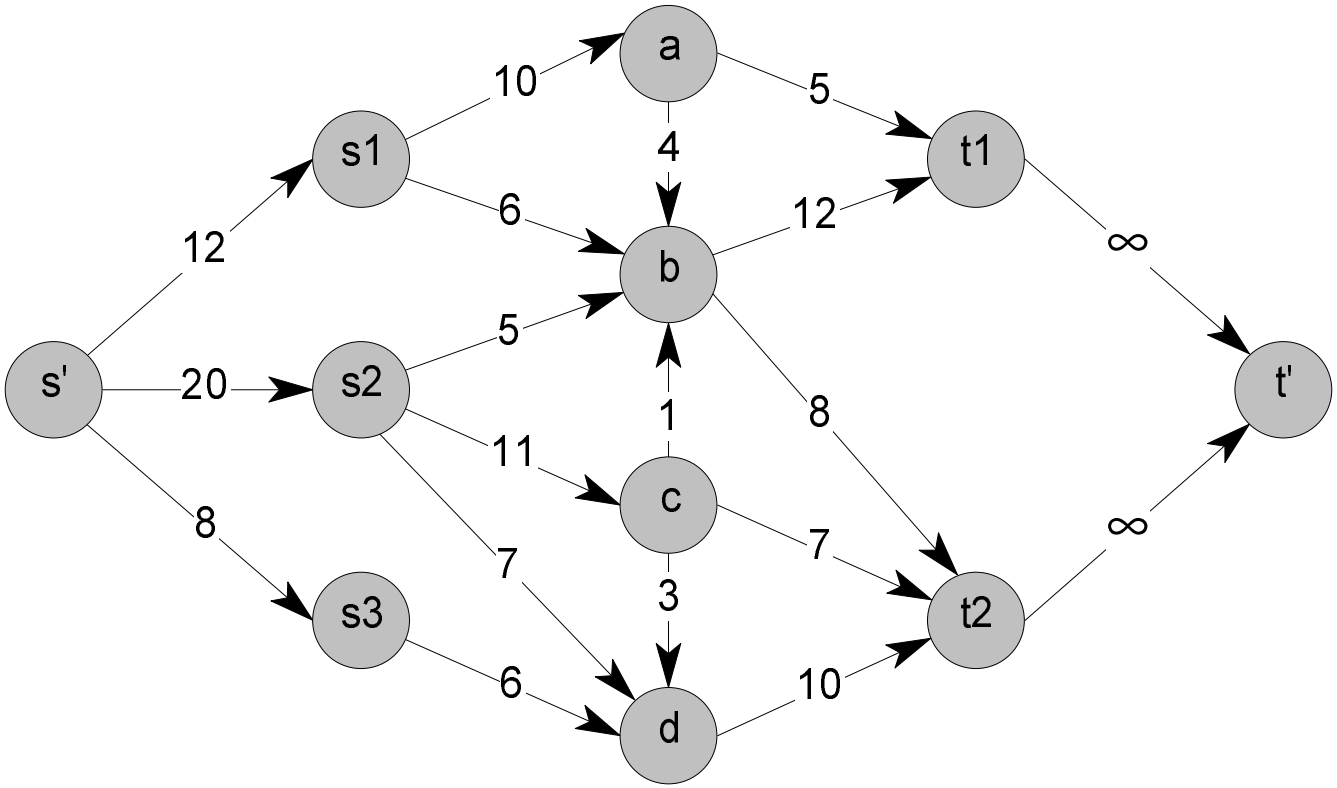




Networks with multiple sources and sinks. Limited source capacity.



Networks with multiple sources and sinks. Limited source capacity.



Outline

Minimum Cost Maximum Flow Problem

Definition

A **flow network with costs** is an ordered 5-tuple $N_d = (G, s, t, c, d)$, where $G = (V, A)$ is a digraph, **source** $s \in V$ and **sink** $t \in V$ are two distinct vertices, c — is a **capacity** function $c\colon V \times V \to \mathbb{R}_+ \cup \{0\}$ satisfying $c_{uv} > 0 \Rightarrow (u, v) \in A$, and d — a **cost** function $d\colon V \times V \to \mathbb{R}$, satisfying $d_{uv} \neq 0 \Rightarrow (u, v) \in A$.

The concept of flow is defined in the same way as for ordinary flow networks. For flow f its **cost** is defined by

$$D = \sum_{uv \in A} f^+_{uv} d_{uv}.$$

Problem: find the maximum flow with minimum cost.

Minimum Cost Maximum Flow Problem

Definition

A **flow network with costs** is an ordered 5-tuple $N_d = (G, s, t, c, d)$, where $G = (V, A)$ is a digraph, **source** $s \in V$ and **sink** $t \in V$ are two distinct vertices, c — is a **capacity** function $c\colon V \times V \to \mathbb{R}_+ \cup \{0\}$ satisfying $c_{uv} > 0 \Rightarrow (u, v) \in A$, and d — a **cost** function $d\colon V \times V \to \mathbb{R}$, satisfying $d_{uv} \neq 0 \Rightarrow (u, v) \in A$.

The concept of flow is defined in the same way as for ordinary flow networks. For flow f its **cost** is defined by

$$D = \sum_{uv \in A} f^+_{uv} d_{uv}.$$

Problem: find the maximum flow with minimum cost.

The concept of residual network . . .

is almost identical to the usual flow networks. One has to be careful about the costs and **opposite arcs** (figure).

Busacker–Gowen Algorithm

This is an application of the Ford–Fulkerson method

Function Flow_BG(G, s, t, c, d);
  $f := 0$;
  while there is a path $s \longrightarrow t$ in residual network **do**
    $p :=$ augmenting path;
    $c :=$ capacity of path p;
    augment flow f along path p (by c);
  Flow_BG $:= f$;

As an augmenting path, the cheapest path $s \longrightarrow t$ should be chosen (this can be done via the Bellman–Ford algorithm for shortest paths).

Busacker–Gowen Algorithm

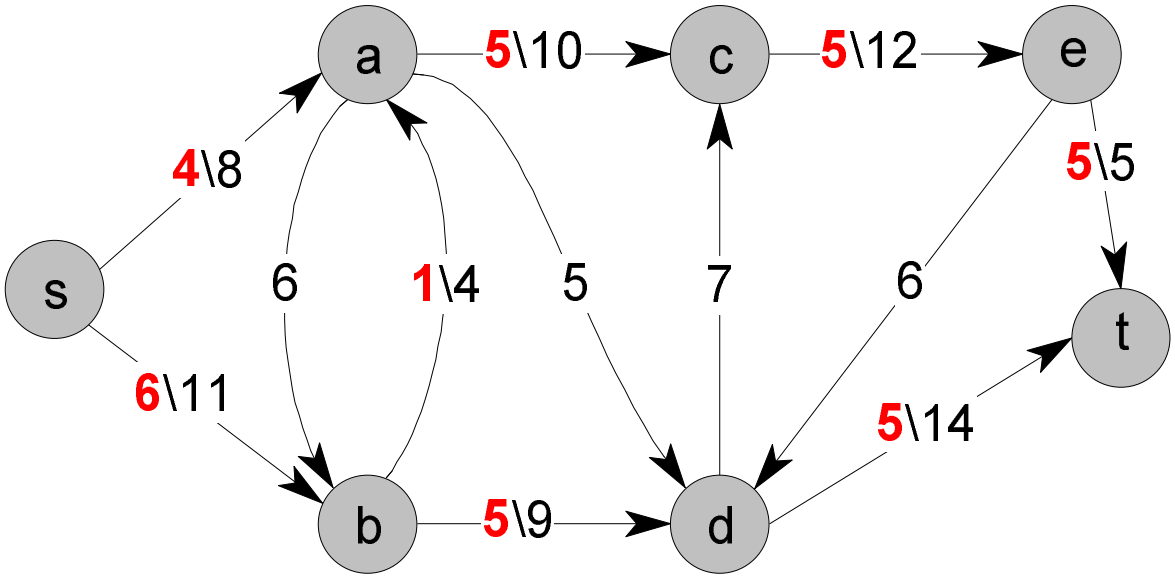
This is an application of the Ford–Fulkerson method

Function Flow_BG(G, s, t, c, d);
  $f := 0$;
  while there is a path $s \longrightarrow t$ in residual network **do**
    $p :=$ augmenting path;
    $c :=$ capacity of path p;
    augment flow f along path p (by c);
  Flow_BG $:= f$;

As an augmenting path, the cheapest path $s \longrightarrow t$ should be chosen (this can be done via the Bellman–Ford algorithm for shortest paths).

Outline

Network flow application



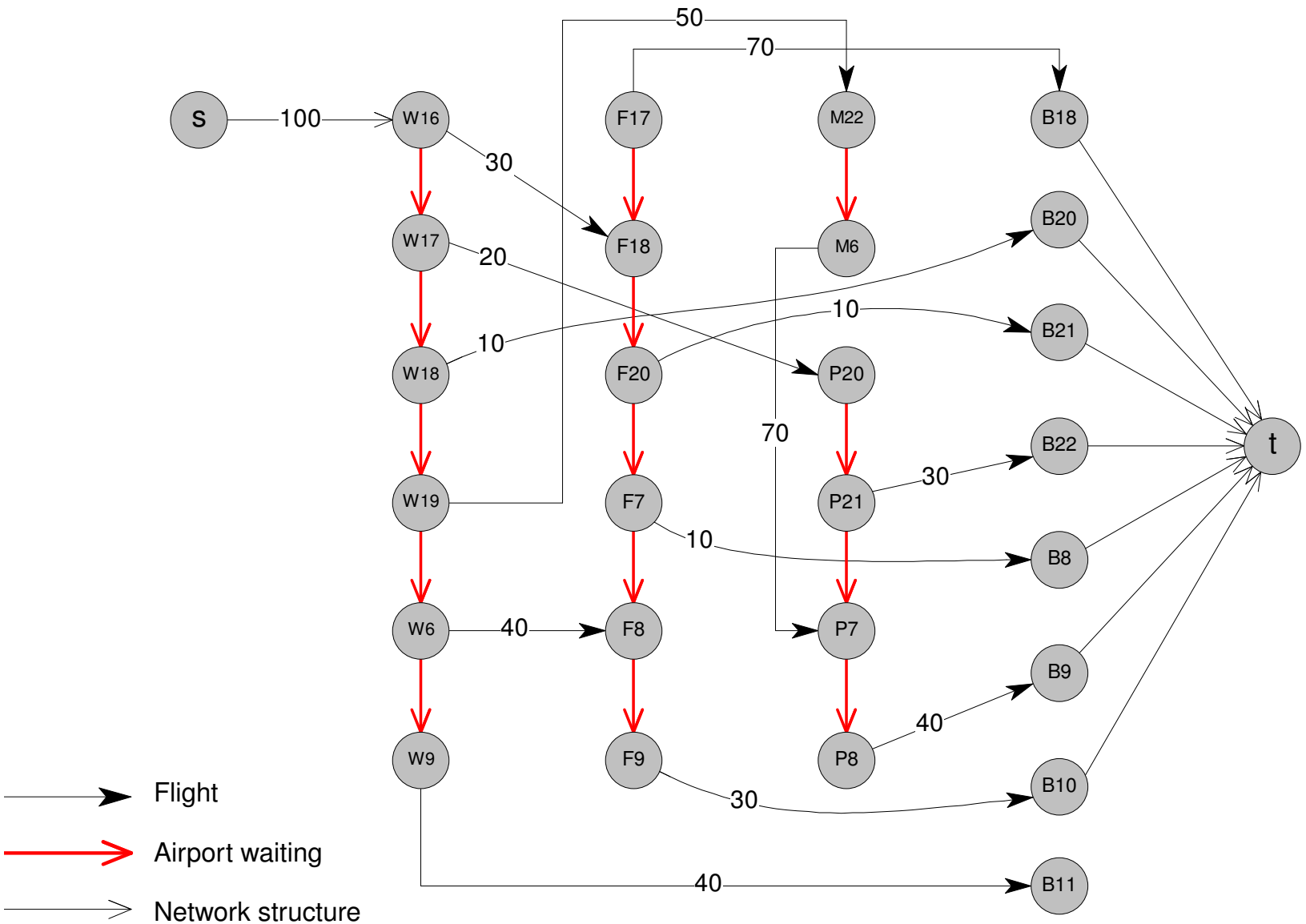
Flight ticket booking problem

Let's say we have a task to organize a flight for a group of 100 people between Warsaw and Brussels, for some conference. The participants can fly out of Warsaw at 4:00 PM at the earliest and should land in Brussels at 10:00 AM the following day at the latest. Unfortunately, there are no more seats available on the direct flight, so we will have to change planes. Knowing the flight schedule and the number of seats available on the planes, determine whether everyone will make it to the conference and specify which tickets should be booked.

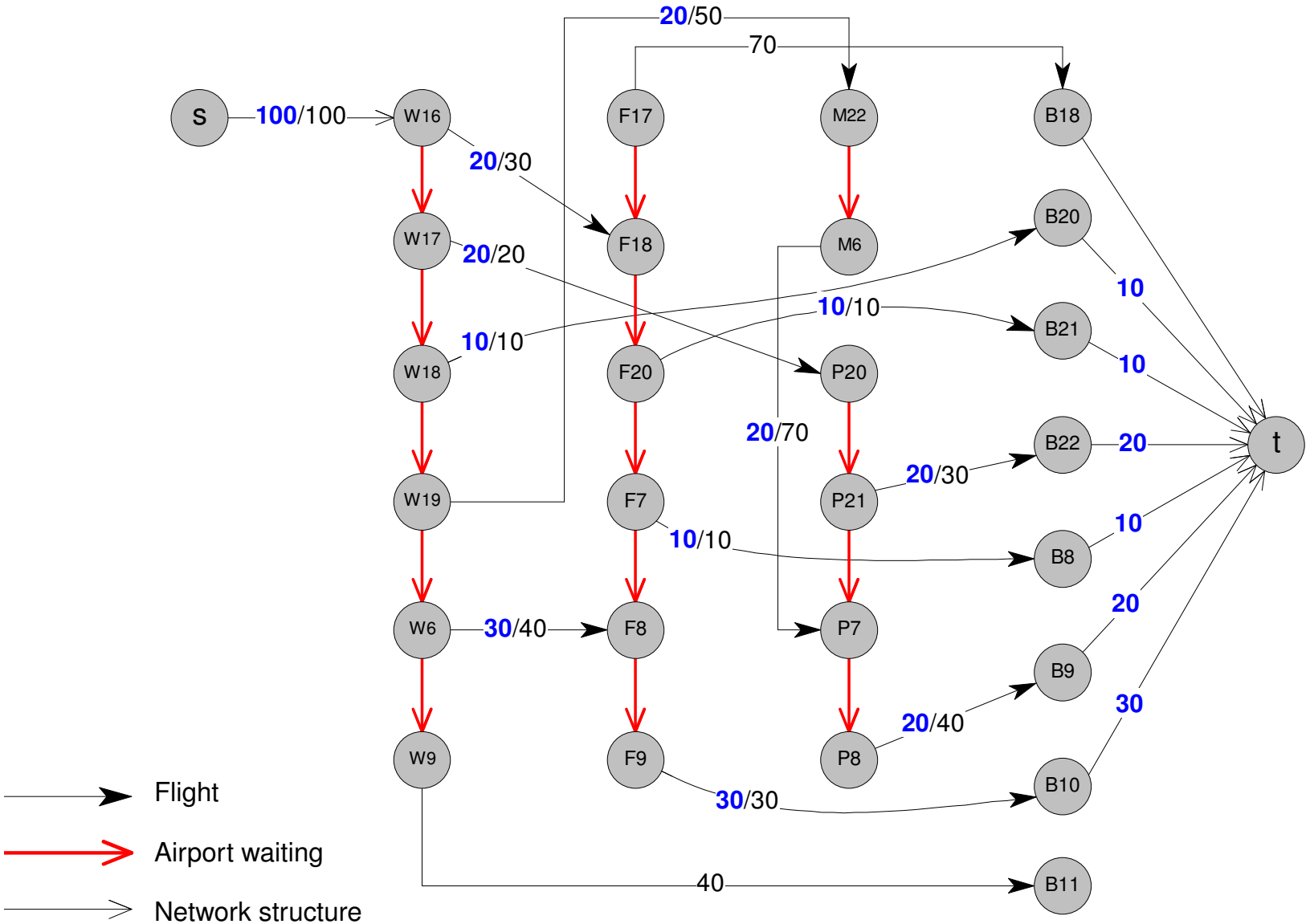
Flight ticket booking problem

Starting Airport	Destination Airport	Departure time	Arrival time	Number of seats
Warsaw	Brussels	18:00	20:00	10
		9:00	11:00	40
Warsaw	Frankfurt	16:00	18:00	30
		6:00	8:00	40
Warsaw	Paris	17:00	20:00	20
Warsaw	Madrid	19:00	22:00	50
Frankfurt	Brussels	17:00	18:00	70
		20:00	21:00	10
		7:00	8:00	10
		9:00	10:00	30
Paris	Brussels	21:00	22:00	30
		8:00	9:00	40
Madrid	Paris	6:00	7:00	70

Flight ticket booking problem — a flow network



Flight ticket booking problem — a flow network

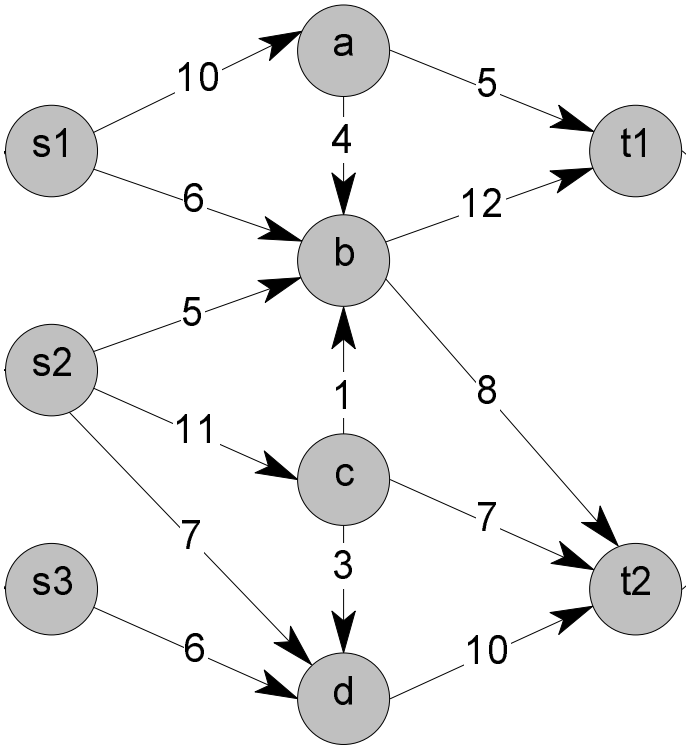


Flight ticket booking problem

But this is probably not the only group that wants to fly somewhere at this time.

Flight ticket booking problem

But this is probably not the only group that wants to fly somewhere at this time.

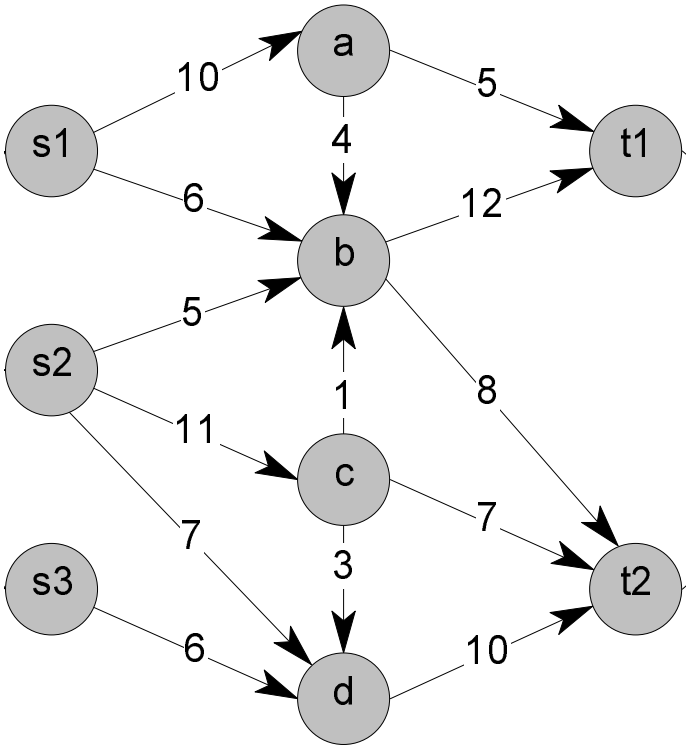


Instead of one flow, we can consider multiple flows: $f_1, \ldots, f_k$, with common capacity conditions:

$$\text{i.e. } \sum_{i=1}^{k} f_i(u,v) \leq c(u,v)$$

Flight ticket booking problem

But this is probably not the only group that wants to fly somewhere at this time.



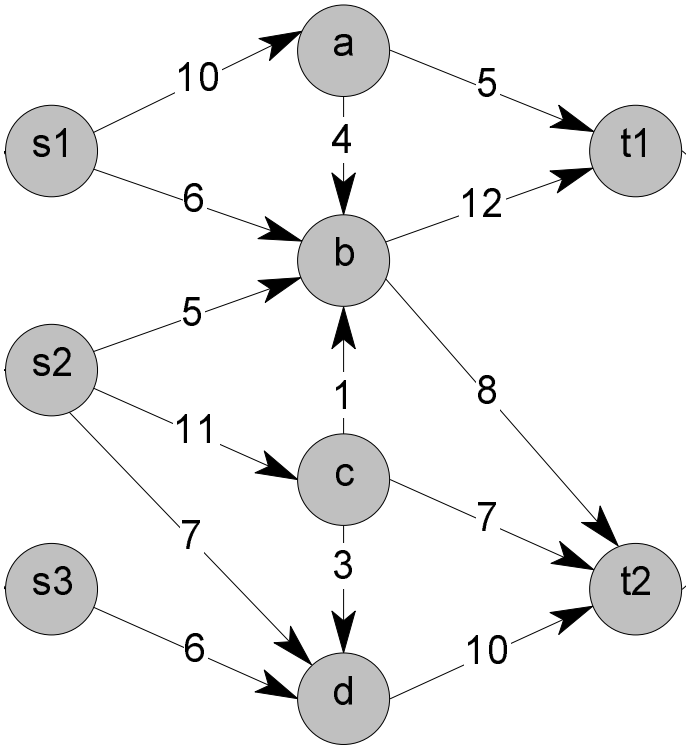
Instead of one flow, we can consider multiple flows: $f_1, \ldots, f_k$, with common capacity conditions:

$$\text{i.e. } \sum_{i=1}^{k} f_i(u,v) \leq c(u,v)$$

What to optimize then?

Flight ticket booking problem

But this is probably not the only group that wants to fly somewhere at this time.



Instead of one flow, we can consider multiple flows: $f_1, \ldots, f_k$, with common capacity conditions:

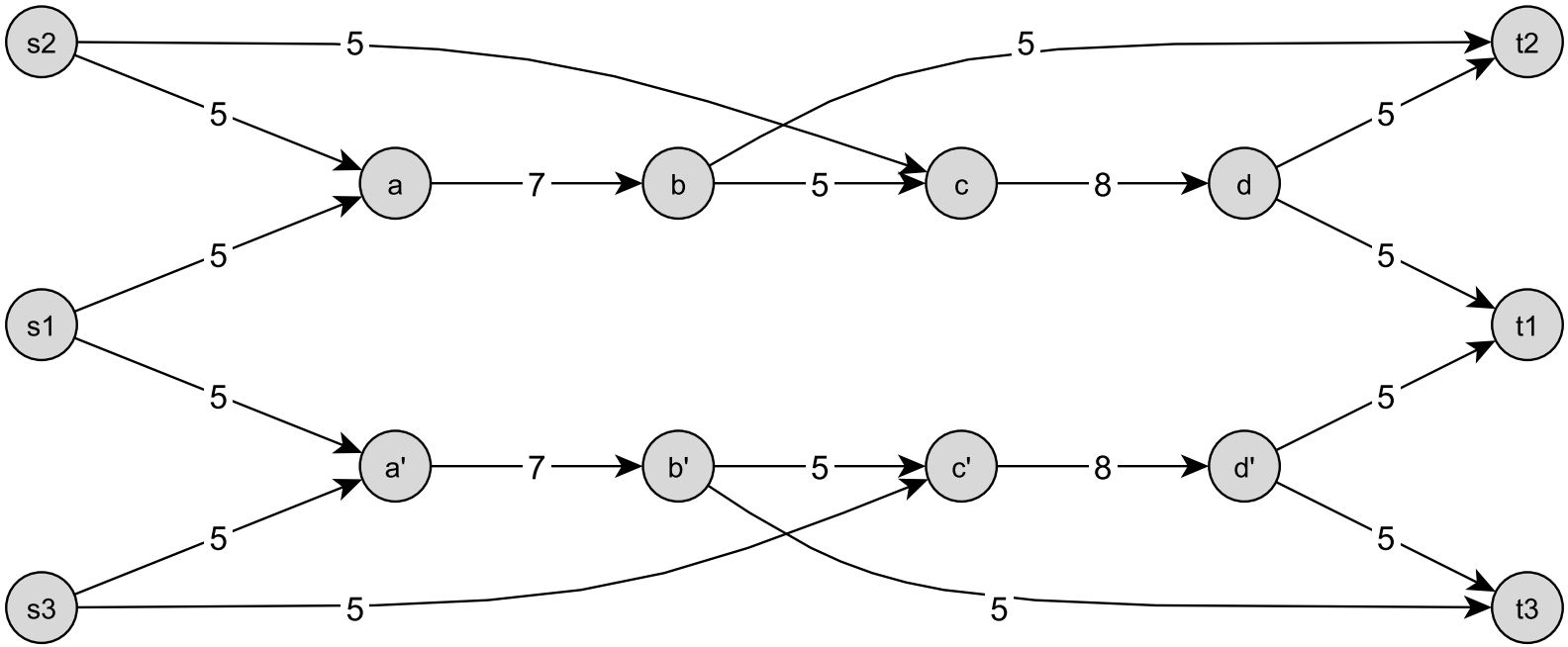
$$\text{i.e. } \sum_{i=1}^{k} f_i(u,v) \leq c(u,v)$$

Meeting the demands?

$$D = \begin{array}{|c|c|c|} \hline & t_1 & t_2 \\ \hline s_1 & 9 & 6 \\ s_2 & 6 & 10 \\ s_3 & 0 & 6 \\ \hline \end{array}$$

Where is the difficulty?

This is linear programming, so it is supposedly easy, but . . .



$$D = \begin{array}{|c|c|c|c|} \hline & t_1 & t_2 & t_3 \\ \hline s_1 & 7 & - & - \\ s_2 & - & 8 & - \\ s_3 & - & - & 8 \\ \hline \end{array}$$

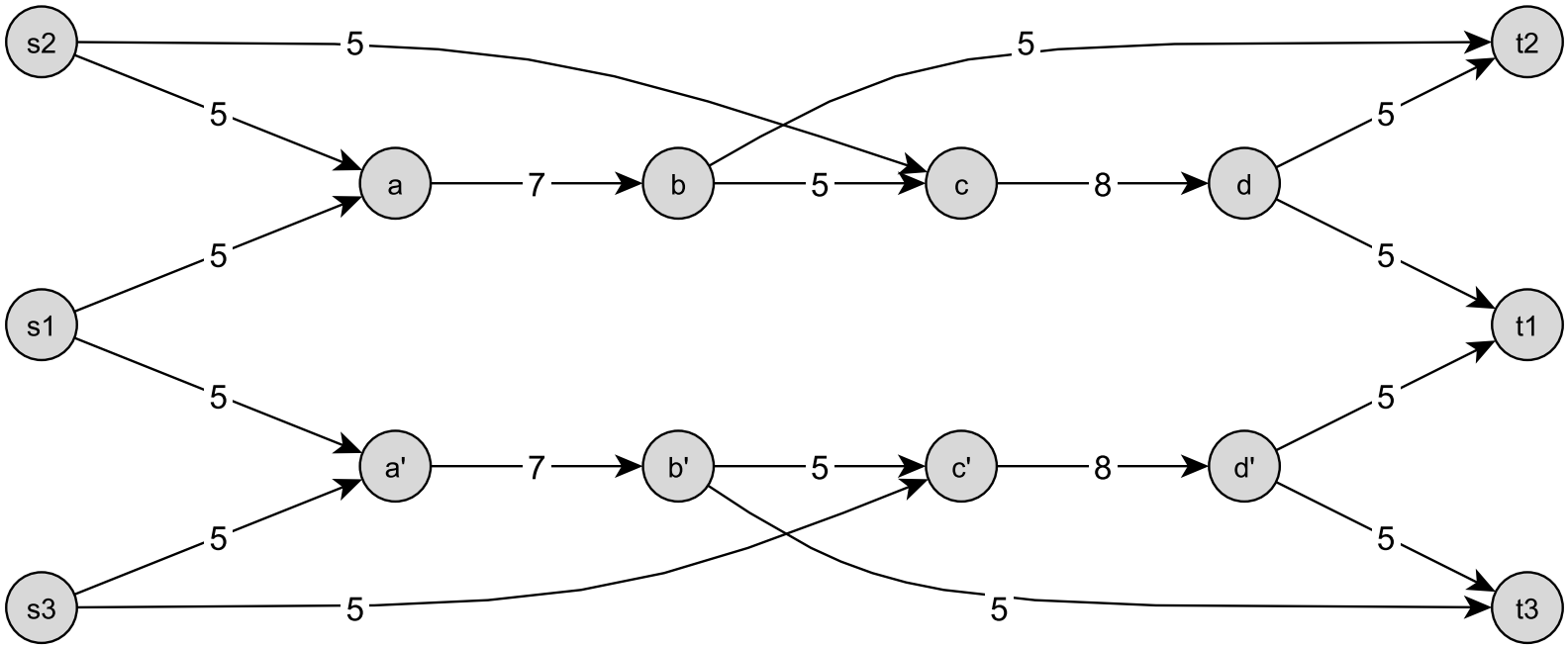
$$\max \sum_i f_i(V, t_i)$$

with flow conditions and

$$f_i(s_i, V) \leq d_{ii}, \; f_i(V, t_i) \leq d_{ii}$$

Where is the difficulty?

This is linear programming, so it is supposedly easy, but . . .



$$D = \begin{array}{c|ccc} & t_1 & t_2 & t_3 \\ \hline s_1 & 7 & - & - \\ s_2 & - & 8 & - \\ s_3 & - & - & 8 \end{array}$$

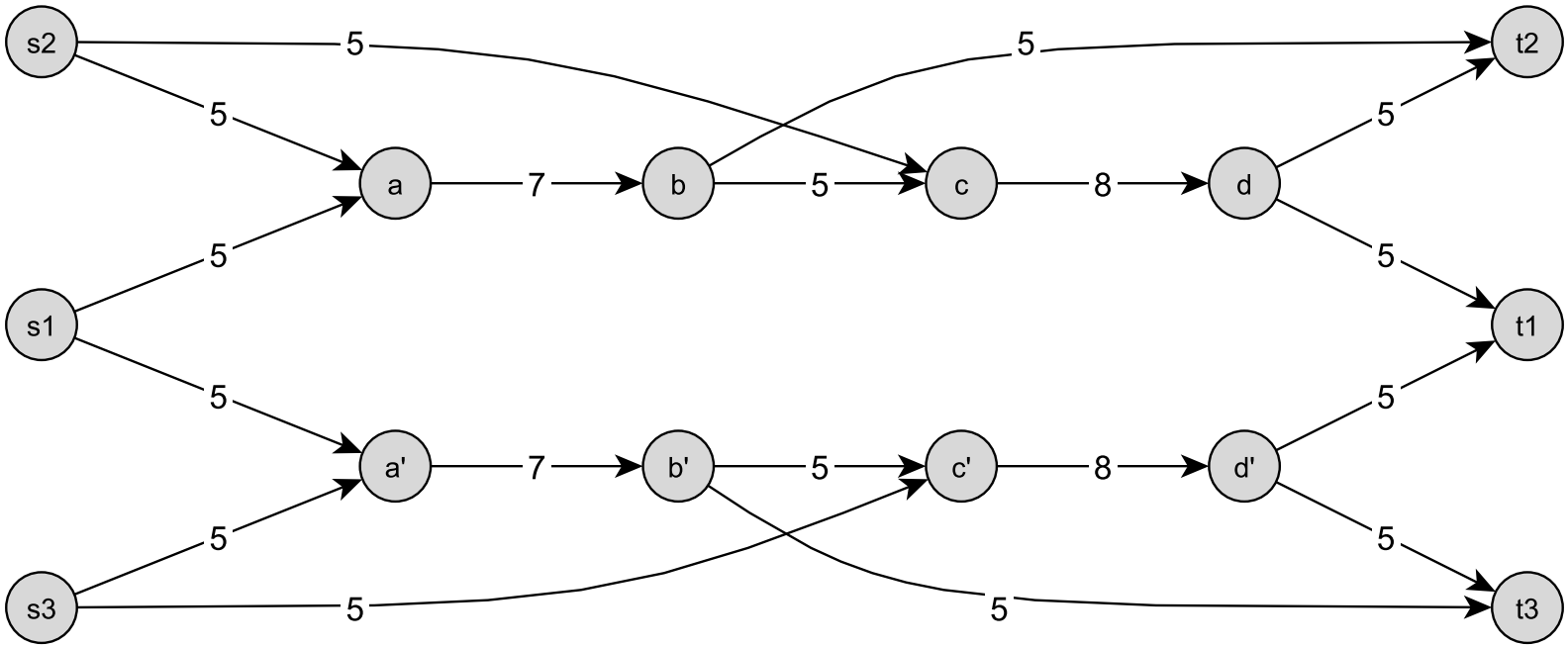
$$\max \sum_i f_i(V, t_i)$$

with flow conditions and

$$f_i(s_i, V) \leq d_{ii}, \; f_i(V, t_i) \leq d_{ii}$$

Where is the difficulty?

This is linear programming, so it is supposedly easy, but . . .



$$D = \begin{array}{|c|c|c|c|} \hline & t_1 & t_2 & t_3 \\ \hline s_1 & 7 & - & - \\ s_2 & - & 8 & - \\ s_3 & - & - & 8 \\ \hline \end{array}$$

$$\max \sum_i f_i(V, t_i)$$

with flow conditions and

$$f_i(s_i, V) \leq d_{ii},\ f_i(V, t_i) \leq d_{ii}$$

the integer value of f_i

Back to Integer Linear Programming

Integer linear programming is (as we already know) an *NP*-hard problem. Similarly, the problem of multicommodity flow — that is, there most likely is no efficient algorithm to solve this problem.

So why doesn't the ordinary (single-commodity) flow have this problem?

Back to Integer Linear Programming

Integer linear programming is (as we already know) an *NP*-hard problem. Similarly, the problem of multicommodity flow — that is, there most likely is no efficient algorithm to solve this problem.

So why doesn't the ordinary (single-commodity) flow have this problem?

Back to Integer Linear Programming

Integer linear programming is (as we already know) an *NP*-hard problem. Similarly, the problem of multicommodity flow — that is, there most likely is no efficient algorithm to solve this problem.

So why doesn't the ordinary (single-commodity) flow have this problem?

The maximum flow and minimum cut Theorem (as well as the Ford-Fulkerson algorithm of augmenting paths) guarantee that when the capacity constraints are integer, then an integer solution exists.